



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
Rajiv Gandhi University of Knowledge Technologies – Nuzvid,
Eluru District, Andhra Pradesh – 521202

**Web Application for Malaria Detection with Microscopic
Blood Images Using Yolo Model**

A MAJOR PROJECT REPORT

Submitted in partial fulfillment of requirements for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

by

P.C. AVINASH KUMAR REDDY N190700

P. SAI BHAVANI N190660

M. SHAKUNTHALA N190684

B. SIRISHA N190641

K. MAHESH N190686

Under the esteemed guidance of

Mr. Krishna Kumar Singh



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Rajiv Gandhi University of Knowledge Technologies – Nuzvid,

Eluru District, Andhra Pradesh - 521202

CERTIFICATE OF COMPLETION

This is to certify that the work entitled, “Web Application for Malaria Detection with Microscope Blood Images using Yolo Model” is the bonafied work of **P.C. Avinash Kumar Reddy (IDNo: N190700)**, **P. Sai Bhavani (IDNo: N190660)**, **M.Shakunthala (IDNo: N190684)**, **B.Sirisha (IDNo: N190641)**, **K.Mahesh (IDNo: N190686)** carried out under our guidance and supervision for 4th year major project of Bachelor of Technology in the department of Computer Science and Engineering under RGUKT IIIT, Nuzvid. This work is done during the academic session July 2024 – April 2025, under our guidance.

Mr. Krishna Kumar Singh

Assistant professor,
Department of CSE,
RGUKT, Nuzvid.

Mrs. S.Bhavani

Head of the Department,
Department of CSE,
RGUKT, Nuzvid.



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Rajiv Gandhi University of Knowledge Technologies – Nuzvid,

Eluru District, Andhra Pradesh - 521202

CERTIFICATE OF EXAMINATION

This is to certify that the work entitled, “Web Application for Malaria Detection with Microscope Blood Images using Yolo Model” is the bonafied work of **P.C. Avinash Kumar Reddy (IDNo: N190700), P. Sai Bhavani (IDNo: N190660), M.Shakunthala (IDNo: N190684), B.Sirisha (IDNo: N190641), K.Mahesh (IDNo: N190686)** and here by accord our approval of it as a study carried out and presented in a manner required for its acceptance in the 4th year of Bachelor Of Technology for which it has been submitted. This approval does not necessarily endorse or accept every statement made, opinion expressed or conclusion drawn, as recorded in this thesis. It only signifies the acceptance of this thesis for the purpose for which it has been submitted.

Mr. Krishna Kumar Singh

Assistant professor,
Department of CSE,
RGUKT, Nuzvid.

Project Examiner

RGUKT, Nuzvid.



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Rajiv Gandhi University of Knowledge Technologies – Nuzvid,

Eluru District, Andhra Pradesh - 521202

DECLARATION

We **P.C. Avinash Kumar Reddy (IDNo: N190700), P. Sai Bhavani (IDNo: N190660), M.Shakunthala (IDNo: N190684), B.Sirisha (IDNo: N190641), K.Mahesh (IDNo: N190686)** hereby declare that the project report entitled “Web Application for Malaria Detection with Microscope Blood Images using Yolo Model” done by us under the guidance of Mr.Krishna Kumar Singh, Assistant Professor, is submitted for the fulfillment of a major project during the academic session July 2024 - April 2025 at RGUKT-Nuzvid. We also declare that this project is a result of our own effort and has not been copied or imitated from any source. Citations from any websites are mentioned in the references. The results embodied in this project report have not been submitted to any other university or institute for the award of any degree or diploma.

Date : 02-05-2025

Place : Nuzvid

P.C. AVINASH KUMAR REDDY N190700

P. SAI BHAVANI N190660

M. SHAKUNTHALA N190684

B.SIRISHA N190641

K.MAHESH N190686

ACKNOWLEDGEMENT

We would like to express our sincere gratitude and appreciation to all those who have contributed to the completion of this mini project on Autism Spectrum Disorder detection using machine learning techniques.

First and foremost, we extend our heartfelt thanks to **Mr. Krishna Kumar Singh sir** whose guidance, support, and expertise were invaluable throughout the project. Their insights and feedback significantly enriched our understanding and guided us in the right direction.

We express our gratitude to **Mrs. S.Bhavani (HOD of CSE)** and other faculty members for being source of inspiration and constant encouragement which helped us in completing the project successfully.

An Endeavour over long period can also be successful by constant effort and encouragement. We wish to take this opportunity to express our deep gratitude to all the people who have extended their cooperation in various ways during our project work. It is our pleasure to acknowledge the help of all those respected individuals.

Project Associates

P.C. AVINASH KUMAR REDDY	N190700
P. SAI BHAVANI	N190660
M. SHAKUNTHALA	N190684
B.SIRISHA	N190641
K.MAHESH	N190686

Table of Contents

S. NO	INDEX	PAGE NO
	Abstract	7
	List of Figures	8
	List of Tables	8
1.	Introduction	9
	1.1 Problem Statement	10
	1.2 Motivation	10
	1.3 Contribution	11
2.	Literature Review	12
3.	Modules	15
	Module 1: YOLO (You Only Look Once) Model	15
	Approach	
	Module 2: Dataset Collection and Preparation	18
	Module 3: Architectural Flow Diagram	20
	Module 4: Code	21
	Module 5: Loss functions of Yolo	28
4.	Results and Discussions	29
5.	Conclusion and Future Enhancements	33
	References	35

ABSTRACT

Malaria is a life-threatening disease caused by the Plasmodium parasite, primarily transmitted through the bites of infected Anopheles mosquitoes. Early and accurate diagnosis is critical for effective treatment and disease control. This project presents a web-based malaria detection system utilizing deep learning-based object detection techniques. By employing the YOLOv8 model, the system processes microscopic blood smear images to automatically detect and classify different cell types, including red blood cells, trophozoites, schizonts, rings, leukocytes, and difficult cases.

Traditional malaria diagnostic methods, such as microscopic examination and rapid diagnostic tests (RDTs), though widely used, are limited by the need for skilled personnel, expensive laboratory equipment, and the risk of inaccurate results in low-resource settings. While machine learning-based solutions have been explored, many lack effective online deployment, limiting their accessibility.

To overcome these challenges, the proposed web application integrates a YOLOv8-trained object detection model for real-time analysis of uploaded blood smear images. Users can upload images through an intuitive web interface and receive immediate detection results. The model is optimized for web deployment to ensure fast and efficient processing. Additionally, the system provides detailed performance evaluation through precision, recall, and F1-score metrics for each cell type. This approach enhances the accessibility, affordability, and speed of malaria diagnosis, offering a practical solution for healthcare providers, particularly in remote and resource-constrained regions.

LIST OF FIGURES

Figure No	Title	Page No
Fig 3.1	Architectural Flow Diagram	20
Fig 5.1	Uploaded Image	29
Fig 5.2	Detected Cells	30
Fig 5.3	Summary of Detection	30
Fig 5.4	Performance Metrics	32
Fig 5.5	YOLOv8 Confusion Matrix	32

LIST OF TABLES

Table No	Title	Page No
Table 2.1	Literature Review	12

CHAPTER 1

INTRODUCTION

The web-based malaria detection system presents a comprehensive study on its development and deployment utilizing deep learning-based object detection techniques. The primary objective is to employ the YOLOv8 model to analyse microscopic blood smear images and accurately detect and classify both malaria-infected and uninfected cells[1]. By integrating computer vision and web technologies, this solution aims to provide an accessible, cost-effective, and real-time diagnostic tool, eliminating dependence on expensive laboratory equipment and highly specialized personnel. The system's implementation focuses on enhancing healthcare accessibility, particularly in remote and resource-constrained regions.

The malaria detection system is developed using Python for model training and optimization, while Ultralytics YOLOv8 and OpenCV libraries facilitate image pre-processing and object detection tasks. Users interact with the system through a Streamlit-based web application, where microscopic blood smear images can be uploaded for analysis. By applying image enhancement techniques such as contrast adjustment and noise reduction, the model's detection performance is improved. The trained YOLOv8 model processes images in real-time, providing accurate identification of six classes: red blood cells, trophozoites, schizonts, rings, leukocytes, and difficult cases.

Existing malaria diagnostic methods, such as microscopic examination and rapid diagnostic tests (RDTs), although widely practiced, face significant challenges. Microscopy, while accurate, requires skilled professionals and laboratory setups, making it less feasible in low-resource environments. RDTs offer quicker results but often lack sufficient sensitivity and specificity, leading to possible misdiagnoses[2]. Moreover, earlier machine learning-based approaches have typically lacked easy online deployment, reducing their real-world practicality.

To overcome these limitations, the proposed web-based malaria detection system integrates a high-performing deep learning model into an interactive web platform, enabling real-time, reliable malaria detection without requiring advanced medical expertise. The system's performance is evaluated using metrics such as precision, recall, F1-score, and inference time, offering a holistic assessment of its effectiveness.

1.1 Problem Statement:

The **web-based malaria detection system** faces several challenges in accurately diagnosing malaria using deep learning techniques [3]. Traditional diagnostic methods, such as microscopic examination and rapid diagnostic tests (RDTs), suffer from limitations including the need for skilled professionals, high costs, and inconsistent sensitivity, leading to delays in treatment and potential misdiagnoses. These issues are particularly concerning in remote and resource-limited regions where access to healthcare infrastructure is minimal.

To tackle these challenges, the malaria detection system leverages the **YOLOv8 object detection model** to analyse blood smear images captured via smartphone cameras. This approach enhances accessibility and affordability while improving diagnostic accuracy. However, the system must also address variations in image quality due to differences in staining techniques, lighting conditions, and smartphone camera specifications. Additionally, optimizing the deep learning model for **web deployment** is crucial to ensure efficient processing without compromising performance.

Addressing the above challenges is essential to enhance the reliability of malaria diagnosis, reduce dependency on laboratory-based testing, and provide a scalable solution for real-time disease detection. By refining the model's accuracy, improving computational efficiency, and ensuring robustness across diverse imaging conditions, the system can play a significant role in combating malaria, particularly in underserved areas.

1.2 Motivation:

Malaria remains one of the most serious infectious diseases worldwide, especially in remote and low-resource areas. Traditional diagnostic methods, such as microscopic examination and rapid diagnostic tests (RDTs), have limitations. Microscopy, though highly accurate, requires skilled technicians and sophisticated laboratory infrastructure, making it difficult to implement in rural regions. RDTs, while faster and more convenient, often suffer from lower sensitivity and specificity, leading to possible misdiagnoses.

Although machine learning models have been explored for malaria detection, many lacked real-world deployment, particularly in accessible platforms like the web. Most solutions

were not optimized for real-time usage and required heavy computational resources, limiting their usability for healthcare workers and patients outside of clinical settings.

To address these challenges, this project introduces a **web-based malaria detection system** utilizing **YOLOv8** object detection. The system enables users to upload microscopic blood smear images and receive real-time, reliable detection results. By combining deep learning with an easy-to-use web interface, the solution aims to make malaria diagnosis faster, more affordable, and widely accessible, especially in areas where expert medical support is limited.

1.3 Contribution:

The malaria detection system leverages the **YOLOv8 object detection model** to provide a cost-effective, accurate, and efficient solution for diagnosing malaria through smartphone-captured blood smear images. By integrating deep learning techniques with real-time image analysis, the system addresses key challenges in traditional malaria diagnosis, such as high costs, reliance on skilled professionals, and inconsistent diagnostic accuracy.

The system utilizes **Streamlit** for the user interface and **Pillow** for image processing, ensuring seamless image upload and real-time malaria detection. By optimizing the YOLOv8 model for mobile and web deployment, the system provides reliable malaria detection in resource-limited settings without needing cloud infrastructure.

In terms of **performance**, the model achieves strong results across several key metrics, including **accuracy**, **precision**, **recall**, and **F1-score**. These metrics confirm the model's ability to effectively identify malaria-infected cells, minimize false positives and negatives, and deliver fast, consistent, and reliable diagnoses. The system significantly reduces diagnosis time, improving accessibility and efficiency in underserved areas.

This work introduces an innovative approach to malaria detection, setting a new benchmark for efficient, scalable, and affordable disease diagnostics in the mobile health space

CHAPTER 2

LITERATURE REVIEW

Table 2.1: Literature Review

Reference	Authors	Year	Objectives	Contributions	Outcomes	Research Gaps
1	1. Deepali A Ghate 2. Chaya Jadhav	2012	To develop an image processing algorithm for automated malaria detection in Giemsa-stained blood smears.	Introduced an approach using image segmentation and thinning techniques to detect malaria parasites.	Achieved high sensitivity, specificity, and predictive values in parasite detection.	The system may not be as precise as manual counting due to ignoring border cells.
2	1. W. David Pan 2. Yuhang Dong 3. Dongsheng Wu	2018	To classify malaria-infected cells using deep convolutional neural networks (CNNs).	Developed a deep learning-based classification system using LeNet-5 CNN architecture. Used data augmentation techniques to improve classification accuracy.	Achieved high accuracy in detecting malaria-infected cells. Demonstrated that data augmentation in the feature domain is more effective than in the	Limited dataset size led to overfitting risks.

					spatial domain.	
3	1.P.A. Pattanaik 2. Mohit Mittal 3. Mohammad Zubair Khan 4. S.N. Panda	2022	To classify malaria-infected cells using deep residual networks with mobile microscopy.	Developed the Multi-Magnification Deep Residual Network (MM-ResNet) for classifying malaria-infected erythrocytes using smartphone images. Introduced parallelism techniques to reduce computational cost and enhance training efficiency.	Achieved an accuracy outperforming several baseline models like CNN and SVM. Demonstrated that MM-ResNet effectively preserves image information across layers.	Smartphone images have lower resolution than digital microscopy, which can affect classification accuracy. 2. The model's effectiveness on large-scale datasets needs further evaluation.
4	Couros Meharian	2017	To develop a deep learning-based computer vision system for	Introduced a CNN-based system that can identify malaria parasites in field-	Achieved WHO Competency Level 1 for P. falciparum diagnosis.	Limited Sensitivity and Specificity of Rapid Diagnostic Test.

			automated malaria diagnosis and quantitation.	prepared thick blood films. Used global white balance, adaptive nonlinear grayscale, and novel augmentation schemes.	Provided accurate quantitation for drug resistance studies.	
5	1. Kenia Hoyos 2. William Hoyos	2024	To develop a deep learning-based system for malaria diagnosis and parasite quantification.	Used YOLOv8 for parasite and leukocyte detection. Implemented data augmentation to improve model performance.	Reduced diagnosis time compared to human experts.	1. Model Performance depends highly on quality of augmented data. 2. No mobile integration.

CHAPTER 3

MODULES

Module 1: YOLO (You Only Look Once) Model Approach

- For the object detection in deep learning applications, we can employ the YOLO (You Only Look Once) algorithm.
- It refers to a class of algorithms based on artificial neural networks that are optimized to work with unstructured data such as images, voice, videos, and text.
- It provides accurate results and we can easily train and test the model and it also provide various matrix like confusion matrix default.
- We use Yolov8 version in this project and it accepts the data in .yaml form.
- Yolov8 detect the 80 different objects by default.

Step 1: Model Setup and Training

- Library Import:
 - Import essential libraries including Ultralytics YOLOv8, Pillow, OpenCV, Streamlit, and others required for image handling, model training, and deployment.
- Model Selection:
 - A YOLOv8 pre-trained model is chosen for object detection due to its fast convergence and high detection accuracy, even on complex microscopic images.
- Training the Model:
 - Fine-tune the pre-trained YOLOv8 model using the training dataset. This involves setting the correct input size, learning rate, and epoch count (50 epochs in this case) to achieve optimal model performance.
 - During training, the model learns to identify and classify the malaria-infected cells in the images, while minimizing the loss function.
- Model Validation:

- After training, evaluate the model on the validation dataset to check its performance. The validation step is crucial for assessing model accuracy and avoiding overfitting.

Step 2: Building the Web Application Using Streamlit

- Web App Design:
 - Use Streamlit, a Python library, to build the web application interface. The user interface is designed to be simple and intuitive, allowing users to easily upload images and view the results.
- Streamlit Features:
 - The file uploader widget is implemented to allow users to upload JPG, PNG, and JPEG image files.
 - Once an image is uploaded, the image is passed through the YOLOv8 model for detection.

Step 3: Image Preprocessing

- Image Format Conversion:
 - The uploaded image is loaded using PIL (Python Imaging Library) and converted into a format that is compatible with the YOLOv8 model (RGB).
 - Resizing the image may also be performed to match the required input size for the model (typically 640 X 640).

Step 4: Inference and Object Detection

- Model Inference:
 - The pre-processed image is passed through the fine-tuned YOLOv8 model to detect the malaria-infected cells. The model performs object detection by predicting bounding boxes and classifying the detected cells into one of the predefined classes.
- Bounding Boxes and Confidence Scores:

- After detecting the infected cells, bounding boxes are drawn around each detected cell along with the confidence score to indicate the model's certainty in the detection.

Step 5: Classification and Counting of Cells

- **Classifying Detected Cells:**
 - Once the cells are detected, the model classifies them into categories such as trophozoite, schizont, ring and gametocyte.
 - Normal red blood cells (RBCs) and leukocytes are ignored in the classification process, as the focus is on identifying infected cells.
- **Counting Detected Cells:**
 - The model counts the total number of detected malaria-infected cells and summarizes the number of each type of infected cell.

Step 6: Result Visualization and Reporting

- **Displaying Results:**
 - The original image is displayed alongside the detection results, where the bounding boxes highlight the infected cells.
 - A text summary is generated that shows the number and types of detected malaria-infected cells.
- **User-Friendly Interface:**
 - The web app ensures the output is presented in a clear and easy-to-understand format, allowing non-experts to interpret the results effectively.

Module 2: Dataset Collection and Preparation

- **Dataset Selection:**

- The project uses the Kaggle **Malaria Detection dataset**, which contains **microscopic blood smear images** annotated with **bounding boxes** to indicate the location of malaria-infected cells.
- The dataset includes **6 classes**: red blood cell, trophozoite, schizont, ring, leukocyte, and gametocyte, with images being of high resolution and requiring careful pre-processing.
- The dataset comprises a collection of labeled images depicting both uninfected and infected cells. Sourced from kaggle Datasets, it contains a total of 1208 images.
- The dataset contain 1600X1200 size of microscopic blood images.
- Dataset is divided into 80% of images (966) for train and 20% of images (242) for test.
- For each image in train and valid, we have annotation text file containing bounding box info and respective class (i.e. RBCs, leukocytes, rings, trophozoites, schizonts and gametocytes) in yaml format.
- In yaml the data in the form key-value pairs.

- **Dataset Splitting:**

- The dataset is split into two parts: **80% for training** and **20% for validation**. This split ensures that the model is both trained and evaluated effectively.

- **Data Augmentation:**

Libraries

1. Streamlit

- **Usage:**

- Used to create the web application interface.
- Allows users to upload images and view the detection results in real-time.
- Handles the layout of the web page and provides easy-to-use components for building the app (buttons, file uploader, images, etc.).

2. YOLOv8 (Ultralytics)

- **Usage:**
 - The YOLOv8 object detection model is used for detecting and classifying malaria cells in the uploaded blood smear microscopy images.
 - Pre-trained and fine-tuned on the malaria dataset for detecting cells like trophozoites, schizonts, rings, gametocytes.
 - Provides accurate bounding boxes around detected cells.

3. Pillow (PIL)

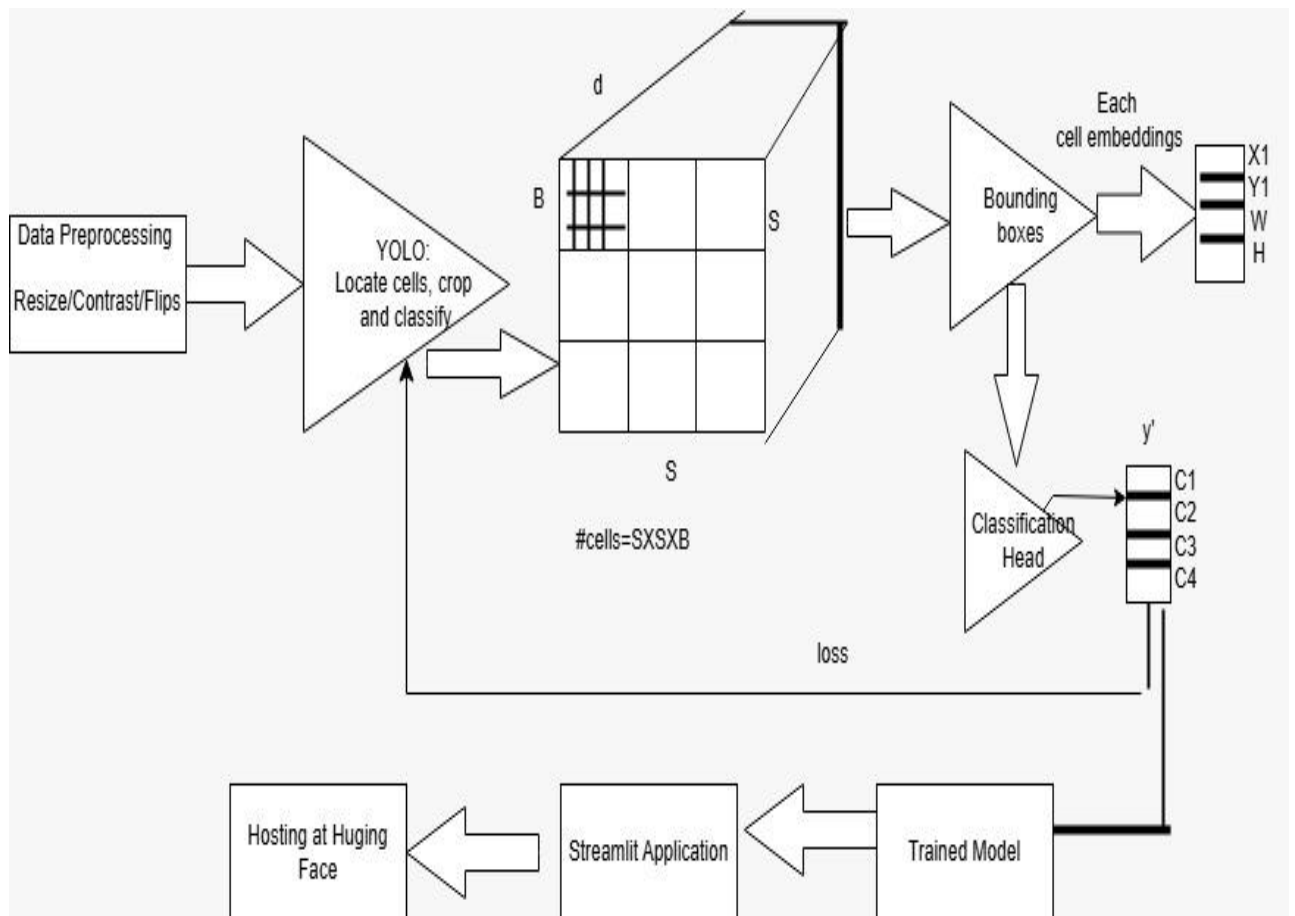
- **Usage:**
 - Used to load and process images before they are fed into the YOLO model.
 - Handles image formats like JPG, JPEG, PNG, and ensures the image is in a compatible format (e.g., RGB).

4. OpenCV

- **Usage:**
 - Utilized for additional image processing tasks like resizing, converting color spaces, or handling specific image manipulations before passing the images to YOLO.
 - Can be used to overlay bounding boxes and labels on the images.

Module 3: Architectural Flow Diagram

Fig 3.1 Architectural Flow Diagram



Module 4: Code

Step 1: YOLO Model

1. Download Malaria Dataset:

- **Dataset Source:** Use the Kaggle "Malaria Detection" dataset, which contains labelled blood smear images with bounding boxes for object detection.
 - Download the dataset from Kaggle (Search for "Malaria Detection").
 - Extract the dataset and organize it into images and their corresponding annotations (bounding boxes).
- **Dataset Link:** <https://www.kaggle.com/datasets/kmader/malaria-bounding-boxes>

2. Download YOLOv8 Pre-trained Model:

Mount Google drive and install YOLO8

This code **mounts your Google Drive** to your Google Colab environment.

- After running it, Colab will ask you to **authorize** access by logging into your Google account.
- Once authorized, your Google Drive files (like datasets, models, etc.) will appear under the /content/drive directory in Colab.
- You can then **read and write files** directly from/to Google Drive — useful for saving models, loading datasets, etc.

```
from google.colab import drive
drive.mount('/content/drive')

!pip install ultralytics --upgrade -q
from ultralytics import YOLO
```

3. Set up paths and class map

This code **prepares the environment** for converting a **Malaria microscopy image dataset** (originally stored in Google Drive) into a **YOLOv8-compatible format**.

Specifically, it:

1. **Imports required libraries** (os, json, shutil)

→ These are needed to handle file system paths, read annotation files, and move/copy files.

2. **Sets up paths:**

→ It defines where the original dataset (training.json and images/) is located inside Google Drive.

3. **Creates output directories:**

→ It creates a new folder structure (malaria_yolo/) where:

- images/train/ will store the images.
- labels/train/ will store the YOLO label files (.txt files).

4. **Defines a class mapping:**

→ It maps each malaria-related cell type (like *trophozoite*, *schizont*, *ring* and *gametocyte*.) to a numeric class ID (needed for YOLOv8 training).

```
import os
import json
import shutil
import random

# Set random seed for reproducibility
random.seed(42)

# Paths
drive_base = "/content/drive/MyDrive/malaria_dataset"
json_path = os.path.join(drive_base, "training.json")
images_path = os.path.join(drive_base, "images")

yolo_base = "/content/drive/MyDrive/malaria_yolo"
for split in ['train', 'val']:
    os.makedirs(f"{yolo_base}/images/{split}", exist_ok=True)
    os.makedirs(f"{yolo_base}/labels/{split}", exist_ok=True)

# Class map
class_map = {
    'red blood cell': 0,
    'trophozoite': 1,
    'schizont': 2,
    'ring': 3,
    'leukocyte': 4,
    'gametocyte': 5
}
```

5. Split the data

- This code splits the dataset `data` into 80% training data and 20% validation data
- `Split_idx` is the index at which to split
- `Train_data` gets the first 80% of the data
- `Val_data` gets remaining 20%

```
split_idx = int(0.8 * len(data))
train_data = data[:split_idx]
val_data = data[split_idx:]
```

4. Converting training.json to YOLOv8 label format

This code **reads** the original **training annotations (JSON file)** and **converts** them into **YOLO format .txt files** for each image, while also **copying the corresponding images** into the new YOLO training folder. After this step, your dataset is now 100% ready to be used for training YOLOv8!

```
def convert_to_yolo(data_split, split_name):
    for entry in data_split:
        img_info = entry['image']
        img_name = os.path.basename(img_info['pathname'])
        w, h = img_info['shape']['c'], img_info['shape']['r']

        label_lines = []
        for obj in entry['objects']:
            cls = obj['category']
            if cls not in class_map:
                continue
            cls_id = class_map[cls]
            box = obj['bounding_box']
            x_min = box['minimum']['c']
            y_min = box['minimum']['r']
            x_max = box['maximum']['c']
            y_max = box['maximum']['r']

            x_center = ((x_min + x_max) / 2) / w
            y_center = ((y_min + y_max) / 2) / h
            width = (x_max - x_min) / w
            height = (y_max - y_min) / h
            label_lines.append(f"{cls_id} {x_center:.6f} {y_center:.6f} {width:.6f} {height:.6f}")

        base = img_name.replace(".png", "")
        with open(f"{yolo_base}/labels/{split_name}/{base}.txt", "w") as f:
            f.write("\n".join(label_lines))

        shutil.copy(os.path.join(images_path, img_name), f"{yolo_base}/images/{split_name}/")

# Convert train and val
convert_to_yolo(train_data, "train")
convert_to_yolo(val_data, "val")
```

5. Write data.yml

This block of code **reads** the annotations from the JSON file, **converts** them into the **YOLO label format**, and **copies** the corresponding images into the appropriate training folder.

```
yaml_content = f"""
path: {yolo_base}
train: images/train
val: images/val

nc: 6
names: ['red blood cell', 'trophozoite', 'schizont', 'ring', 'leukocyte', 'gametocyte']
"""
with open(f"{yolo_base}/data.yml", "w") as f:
    f.write(yaml_content)
```

6. Train Yolo v8

This block of code starts training a YOLOv8 small (yolov8s.pt) model on custom malaria dataset.

```
model = YOLO("yolov8s.pt")
results = model.train(
    data=f"{yolo_base}/data.yml",
    epochs=50,
    imgsz=640,
    batch=8,
    name="malaria_yolo"
)
```


7. Downloading runs file

This part of the code compresses the entire runs folder (which contains YOLOv8 training results) into a ZIP file and downloads it to local computer.

This allows you easily save all training outputs — like trained model weights, evaluation plots, and metrics — from Google Colab.

```
# Step 1: Zip the 'runs' folder
!zip -r runs.zip runs

# Step 2: Download the zip file
from google.colab import files
files.download('runs.zip')
```

8. Evaluate on the Validation set

This block of code loads trained YOLOv8 model and performs evaluation and prediction.

```
from ultralytics import YOLO

# Load trained model
model = YOLO("runs/detect/malaria_yolo/weights/best.pt")

# Run evaluation
metrics = model.val()
```

Step 2: Streamlit Application

1. Importing requirements

Streamlit creates a simple web interface where users can upload images.

PIL handles the image uploads and processes the images for predictions.

YOLO runs the object detection model to predict the locations of malaria cells in images.

Counter helps count the different types of malaria cells detected, which will be displayed to the user in the UI.

```
import streamlit as st
from PIL import Image
from ultralytics import YOLO
from collections import Counter
```

2. Loading Model

This line allows to use the trained YOLOv8 model to make predictions on new images or further evaluate its performance.

```
# Load model
model = YOLO("best.pt")
```

3. App Title and Description

These lines provide a **user-friendly interface** for your app, making it clear to users that the goal is to upload an image, and the model will detect and highlight malaria cells

```
st.title("🦠 Malaria Detection with YOLOv8")
st.write("Upload a microscopy image. The model will highlight detected cells.")
```

4. Uploading Image

This line creates a file uploader widget that lets the user upload an image in either JPEG, PNG, or JPG format.

```
uploaded_file = st.file_uploader("Upload Image", type=["jpg", "jpeg", "png"])
```

5. if uploaded file:

Display Uploaded Image

```
image = Image.open(uploaded_file).convert("RGB")
st.image(image, caption="Uploaded Image", use_column_width=True)
```

6. Perform Inference

```
results = model.predict(image, conf=0.5)
result_img = results[0].plot()
results = results[0] # Get the first result object
```

7. Process Detection Results

After detection, this code filters out normal cells and counts only malaria-infected cells, preparing the data so you can display how many malaria-affected cells are present and what types they are.

```
class_ids = results.bboxes.cls.cpu().numpy().astype(int)
class_names = results.names
counts = Counter(class_ids)

malaria_results = {}
for cls_id, count in counts.items():
    cls_name = class_names[cls_id]
    if cls_name != "red blood cell" and cls_name != "leukocyte":
        malaria_results[cls_name] = count
```

8. Display Detection Image

```
st.image(result_img, caption="Detected Cells", use_column_width=True)
```

9. Display Summary Results

This part generates a clean, readable summary for the user, listing the malaria types and number of affected cells, or informs the user if no malaria infection is detected.

```
if malaria_results:
    summary = ", ".join([f"{name} ({count} cells)" for name, count in malaria_results.items()])
    st.write(f"**Affected with type of malaria named:** {summary}.")
else:
    st.write("No malaria-infected cells detected.")
```

Module 4: Code

Loss Functions of YOLO :

- **Bounding Box Regression Loss (CIoU Loss)**

- Measures the overlap and distance between predicted and ground-truth bounding boxes.

$$\text{CIoU} = \text{IoU} - \frac{\rho^2(\mathbf{c}, \mathbf{c}_{\text{gt}})}{\mathbf{c}_{\text{max}}^2} - \alpha v$$

- **Objectness Loss (Binary Cross Entropy)**

- Measures how confident the model is about the presence of an object in a box.

$$\text{Objectness Loss} = -(y_{\text{obj}} \log(\hat{y}_{\text{obj}}) + (1 - y_{\text{obj}}) \log(1 - \hat{y}_{\text{obj}}))$$

- **Classification Loss (Binary Cross Entropy)**

- Measures the accuracy of class predictions for each bounding box.

$$\text{Classification Loss} = - \sum_{i=1}^C (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i))$$

$$\text{Total Loss} = \text{CIoU Loss} + \lambda_{\text{obj}} \cdot \text{Objectness Loss} + \lambda_{\text{cls}} \cdot \text{Classification Loss}$$

CHAPTER 4

RESULTS AND DISCUSSIONS

When the Input Microscopic blood Image is given it will show as in the Figure 5.1

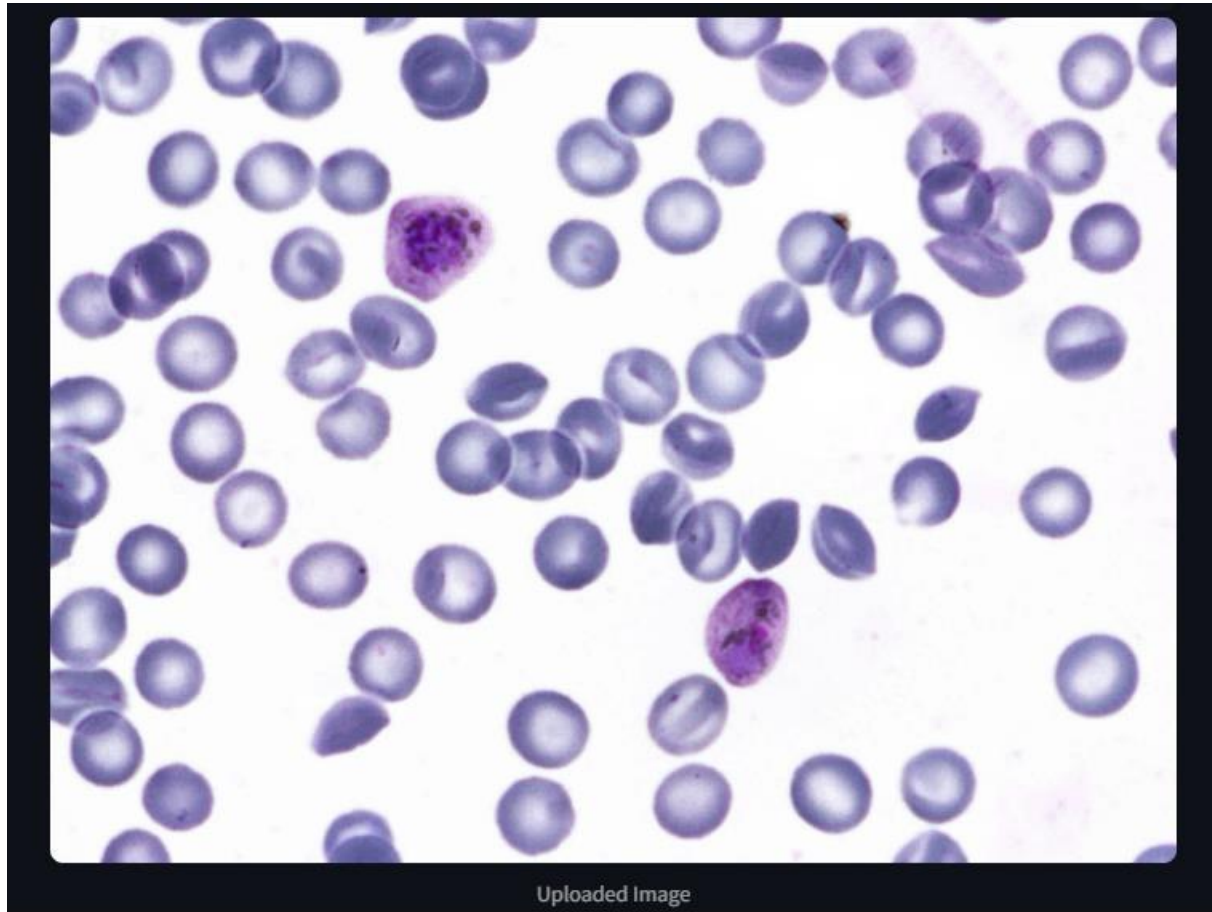


Fig 5.1 : Uploaded Image

After processing the uploaded microscopy image, the model highlights all detected cells directly on the image. Different cell types, including malaria-infected cells (like trophozoites, rings, schizonts) and normal cells (such as red blood cells and leukocytes), are marked with bounding boxes and labelled accordingly.

This annotated detection image as shown in Fig 5.2 provides a clear visual understanding of where and what types of cells have been identified, helping in quickly assessing the presence and severity of malaria infection.



Fig 5.2 : Detected Cells

After analysing the uploaded image, the model counts how many cells of each malaria-infected type (such as trophozoite, ring, gametocyte, schizont) were detected.

The summary section displays the names of the malaria types found along with the number of cells detected for each type. If no malaria-infected cells are found, the app clearly informs the user that no infection was detected in the image as shown in Fig 5.3

Affected with type of malaria named: ring (1 cells), trophozoite (5 cells), gametocyte (1 cells).

Fig 5.3: Summary of Detection

Insights from the Performance Metrics Figure:

- The model detects **red blood cells** and **leukocytes** very accurately with high precision and recall (above 94%).
- **Trophozoites** and **rings** are also detected reasonably well but slightly lower than red blood cells.
- Detection of **schizonts** and **gametocytes** cells is relatively harder for the model, showing lower precision and recall values compared to other classes.

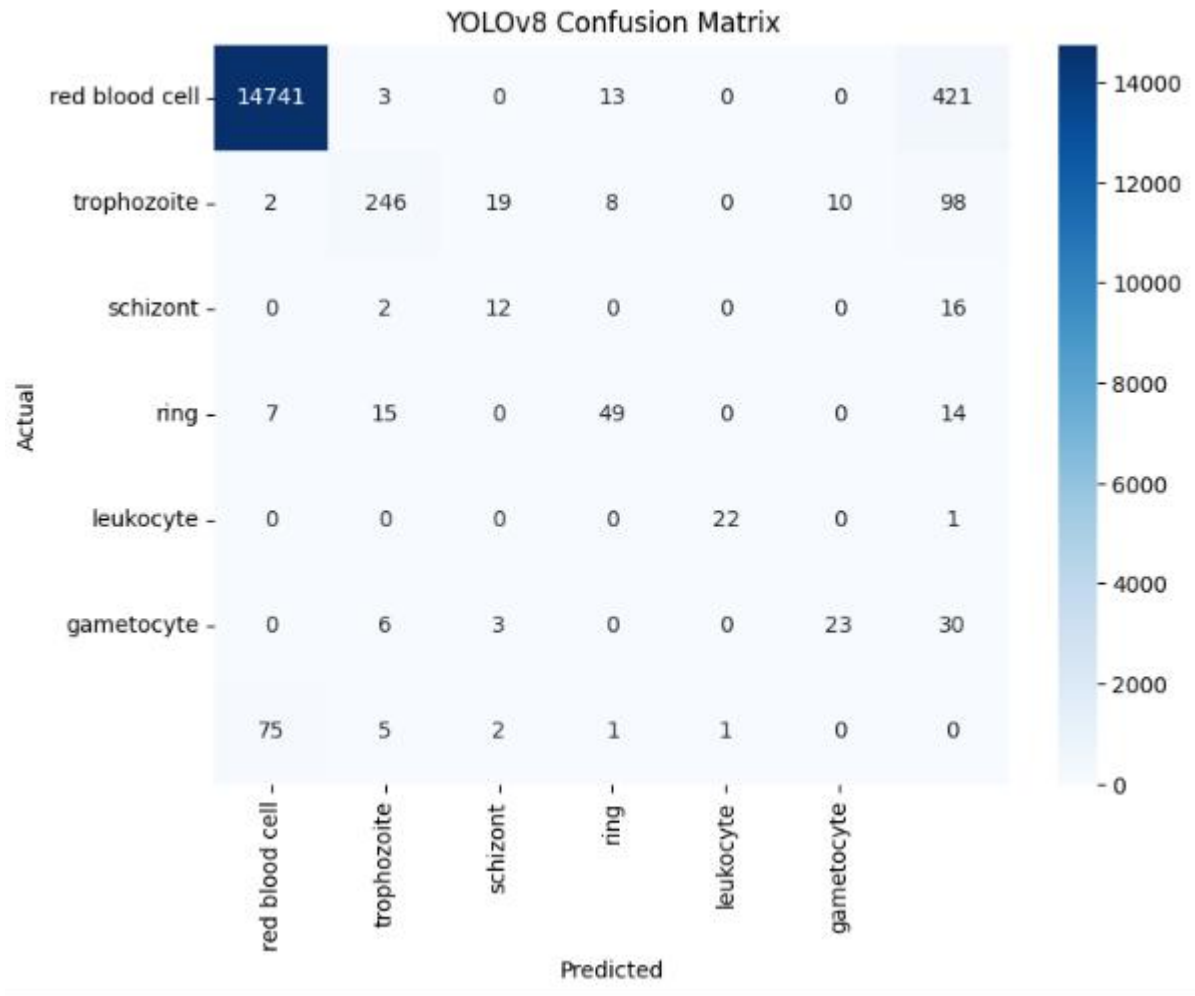


Fig 5.4: YOLOv8 Confusion Matrix

This matrix shown in Fig 5.5 tells how well the YOLOv8 model correctly classified each type of cell. Each row represents the **actual** class, and each column represents the **predicted** class.

Diagonal cells (from top-left to bottom-right) show **correct predictions** (where actual and predicted match).

Off-diagonal cells show **misclassifications** (where the model confused one class for another).

- **Red blood cells** and **leukocytes** are **very accurately** detected.
- **Trophozoites**, **rings**, **schizonts**, and **gametocytes** show **some confusion**, especially among each other.
- Most errors happen between trophozoite, ring, and gametocytes which are visually quite similar.

CHAPTER 5

CONCLUSION AND FUTURE ENHANCEMENTS

Conclusion:

In conclusion, this Malaria Detection project presents the development and implementation of an advanced YOLOv8-based model tailored for the accurate detection and classification of malaria-infected cells in microscopic images. By leveraging the capabilities of state-of-the-art object detection, the system effectively identifies various stages of malaria infection, providing valuable support for medical diagnostics.

The project demonstrates the effectiveness of YOLOv8 in addressing critical challenges in microscopic image analysis, such as distinguishing between similar cell types and detecting small infected regions. Through careful training and evaluation, the model achieves high precision and recall across multiple classes, ensuring reliable and consistent performance.

Compared to traditional diagnostic methods, this automated approach significantly enhances the speed, accuracy, and scalability of malaria detection. The integration of a user-friendly Streamlit interface further promotes accessibility, enabling healthcare professionals and researchers to quickly analyse images and interpret results.

This project contributes meaningfully to the field of medical imaging and disease detection by offering a robust, efficient, and scalable solution for malaria diagnosis. The advancements made in detection accuracy and model reliability pave the way for future innovations, including broader applications in other blood-borne disease identification tasks and improved clinical decision support systems.

Overall, the Malaria Detection with YOLOv8 project represents an important step towards leveraging deep learning for impactful healthcare solutions, ultimately supporting faster diagnosis, better patient outcomes, and strengthened efforts against malaria.

Future Enhancement:

Deploying this application as an Android app will enable easy and widespread access to malaria detection, significantly reducing the cost associated with traditional blood tests, minimizing blood sample wastage, and enhancing the accuracy and efficiency of diagnosis through deep learning techniques.

Incorporate a larger and more diverse set of microscopic images from various geographic regions to improve the model's generalization across different malaria strains and imaging conditions.

Focus on enhancing the model's accuracy for classes with lower recall and precision, such as schizont and difficult cases, by applying targeted data augmentation or advanced re-sampling techniques.

Deploy the model into portable diagnostic devices or mobile applications to enable real-time malaria detection in remote or resource-limited settings.

Optimize the model through pruning, quantization, or lightweight architectures to ensure faster inference and reduced computational demands on mobile or embedded systems.

Expand the model's capabilities to detect and classify other blood-related diseases (e.g., dengue, sickle cell anaemia) within the same microscopic images, making it a more versatile diagnostic tool.

Implement an active learning loop where new images with uncertain predictions are periodically annotated and used to retrain the model, continuously improving its performance over time.

REFERENCES

[1] Jocher, G. et al. (2023). *YOLO by Ultralytics (YOLOv8)*.

<https://github.com/ultralytics/ultralytics>

[2] Wongsrichanalai, C., Barcus, M. J., Muth, S., Sutamihardja, A., & Wernsdorfer, W. H. (2007). *A review of malaria diagnostic tools: Microscopy and Rapid Diagnostic Test (RDT)*.

The American Journal of Tropical Medicine and Hygiene.

https://www.ajtmh.org/view/journals/tpmd/77/6_Suppl/article-p119.xml

[3] W. David Pan, Yuhang Dong, Dongsheng Wu, "Classification of Malaria-Infected Cells Using Deep Convolutional Neural Networks," in *Machine Learning - Advanced Techniques and Emerging Applications*, IntechOpen, 2018.

<https://doi.org/10.5772/intechopen.72426>

[4] Kenia Hoyos, William Hoyos, "Supporting Malaria Diagnosis Using Deep Learning and Data Augmentation," *Diagnostics*, MDPI, Vol. 14, Issue 7, Article 690, March 2024.

<https://doi.org/10.3390/diagnostics14070690>

[5] P.A. Pattanaik, Mohit Mittal, Mohammad Zubair Khan, S.N. Panda, "Malaria Detection Using Deep Residual Networks with Mobile Microscopy," *Journal of King Saud University – Computer and Information Sciences*, Vol. 34, No. 5, pp. 1700–1705, 2022.

<https://doi.org/10.1016/j.jksuci.2020.07.003>

[6] Antora Dev, Mostafa M. Fouda, Leslie Kerby, Zubair Md Fadlullah, "Advancing Malaria Identification from Microscopic Blood Smears Using Hybrid Deep Learning Frameworks," *IEEE Access*, 2022.

<https://doi.org/10.1109/ACCESS.2022.0092316>